

Day2. 인증 서버 구축

1. 수업 내용 정리

클래스 구성 요소

- 필드: 클래스가 가지는 데이터(private으로 캡슐화)
- 생성자: 객체 생성 시 호출(필드 초기화 역할)
- 메소드: 동작/행위 정의(public으로 외부에 제공)
- 접근제어자: public > protected > (default) > private

오버라이딩 vs 오버로딩

오버라이딩: 부모 메소드의 자식을 재정의(@Override)

오버로딩: 한 클래스 안의 메소드가 같은 이름, 다른 매개변수를 가짐.

클래스는 상속은 extends, 인터페이스는 implements의 키워드로 작성.

사용방법 예시:

```
// 상속
public class User {
    // ...
    public User() { }
}

public class AdminUser extends User {
    // ...
    public AdminUser() {
        super(); // 부모 생성자 호출
    }
}
```

```
// 인터페이스
public interface AuthService {
    public void register(String id, String pw) { ... }
}

public class MemoryAuthService implements AuthService {
    @Override register(String id, String pw) { ... } // @Override 키워드
}
```

다형성

하나의 인터페이스 타입으로 여러 구현체를 교체해서 사용하는 능력

- 교체 용이성
- 테스트 용이성

@Override의 역할?

컴파일러에게 오버라이딩 의도 전달 - 메소드명 오타 시 컴파일 오류로 잡아줌

인터페이스, 컬렉션, 예외처리

인터페이스

- 구현 강제: implements 클래스는 인터페이스 안의 모든 메소드를 구현해야 함.
- 다중 구현 가능(implements A, B)
- 타입 선언에 사용: AuthService s = new MemoryAuthService()

컬렉션

- Map: Key-Value 구조
- List: 순서가 있는 목록

예외처리

- RuntimeException 상속으로 커스텀 예외 생성
- throws로 호출자에게 예외 위임
- try-catch: 호출부에서 메시지 출력 처리

JAVA 구조 이해

클래스 = 프로그램 기본 단위

- 클래스 이름과 파일이름을 맞추지 않으면 컴파일 오류 발생

객체 = 클래스를 실제로 사용하는 방식

- JAVA 프로그램은 객체들이 서로 메시지를 주고받는 구조

main = 프로그램 시작점

- JVM이 이 메소드만 직접 호출

패키지 = 클래스들의 논리적 묶음

- 논리적 구조, 디렉토리 구조와 1:1로 매칭 (com.tech.auth.server :: com -> tech -> auth -> server 라는 논리적 공간에 속한다는 뜻)

import = 다른 객체를 가져다 쓰는 선언

- import com.tech.auth.server :: "나는 server라는 클래스를 이 패키지 위치에서 가져다 쓰겠다"라는 의미.
- 디렉토리는 물리적 구조/패키지는 논리적 구조/import는 연결 선언

JDBC vs JPA 차이

JDBC: SQL 직접 작성 / JPA: 객체-DB 자동 매핑 (Java 메소드 호출로 DB 조작)

- 대용량·고성능 트랜잭션이 생명인 환경에서는 실제로 JPA보다 SQL을 직접 짜는 방식(MyBatis, 순수 JDBC) 선호
- JPA는 "실제로 어떤 SQL이 나갈지"를 세밀하게 통제하기 어려움
- 영속성 컨텍스트(1차 캐시)가 대량 처리에서 오히려 불리하게 작용
- 복잡한 동적/집계 쿼리는 SQL이 훨씬 직관적
- 따라서 사용자 상황에 맞게 선별 적용 권고

객체 생성 패턴

- 생성자 패턴: 필드 2~3개, 단순한 경우
 - 장점: 직관적, 빠름 / 단점: 매개변수 많으면 가독성 저하
- 정적 팩토리 메소드: 생성 의미를 명확히 할 때
 - 장점: 이름으로 의도 표현, 캐싱 가능 / 단점: 상속 어려움
- 빌더 패턴: 필드 많고 선택적인 경우
 - 장점: 가독성 최고, 선택 필드 처리 용이 / 단점: 코드량 증가

2. 코드 분석

다음은 Day2 과정에서 받은 loginauth 프로젝트의 코드 분석입니다.

import 살펴보기

```
// ---- JWT(로그인 토큰) 관련 라이브러리 (auth0에서 제공하는 외부 라이브러리) ----
import com.auth0.jwt.JWT; // 토큰을 "생성"할 때 쓰는 빌더
import com.auth0.jwt.JWTVerifier; // 토큰을 "검증"할 때 쓰는 객체
import com.auth0.jwt.algorithms.Algorithm; // 서명 알고리즘(HMAC256 등) 정의
import com.auth0.jwt.exceptions.JWTVerificationException; // 토큰이 위조/만료됐을 때 발생하는 예외
import com.auth0.jwt.interfaces.DecodedJWT; // 검증에 성공한 토큰의 내용(클레임)을 담은 객체

// ---- 순수 자바 표준 라이브러리로 만든 간이 HTTP 서버 (com.sun.net.httpserver) ----
// 스프링부트를 쓰면 내장 톰캣이 이 역할을 대신해주지만, 지금은 이걸 직접 씁니다.
```

```

import com.sun.net.httpserver.HttpExchange; // "요청 1건"을 표현하는 객체 (요청 읽기 + 응답 쓰기를 모두 담당)
import com.sun.net.httpserver.HttpServer;    // 포트를 열고 요청을 받는 서버 본체

// ---- 비밀번호 해시(암호화) 라이브러리 ----
import org.mindrot.jbcrypt.BCrypt; // 비밀번호를 평문 그대로 저장하지 않고 안전하게 해시하기 위한 라이브러리

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress; // "IP주소 + 포트번호" 조합을 표현 (예: 8080 포트에서 대기)
import java.net.URLDecoder;       // form 데이터의 URL 인코딩(%XX)을 원래 문자로 되돌림
import java.nio.charset.StandardCharsets; // 문자 인코딩(UTF-8 등) 상수 모음

// java.sql - DB(MariaDB)와 직접 통신하기 위한 JDBC 표준 API. 필요한 것만 명시(import)함
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.SQLIntegrityConstraintViolationException;
import java.sql.Statement;

// java.time - 날짜/시간을 다루는 최신(Java 8+) 시간 API
import java.time.Instant;
import java.time.temporal.ChronoUnit;

// java.util - 컬렉션(자료구조)과 유틸리티 클래스. 필요한 것만 명시(import)함
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID; // 세션ID처럼 "겹칠 확률이 사실상 0인" 고유 문자열을 만들 때 사용
import java.util.concurrent.ConcurrentHashMap; // 여러 요청이 동시에 접근해도 안전한(thread-safe) Map
import java.util.concurrent.Executors;        // 스레드풀(동시에 여러 요청을 처리할 작업자 묶음)을 생성

/* 파일 전체 구조
 * main() (프로그램 시작점)
 *   | new JdbcUserRepository(...) DB 접속 정보를 들고 있는 객체 생성
 *   | new AuthService(userRepository) ↑ 위 객체를 "주입"받아 로그인/가입 로직 수행
 *   | HttpServer.createContext(...) URL 경로별로 "요청이 오면 실행할 코드(람다)"를 등록
 *   |
 *   | (요청이 들어올 때마다 아래 순서로 호출됨)
 *   |
 *   | handler 람다 (exchange -> { ... })
 *   |   | parseFormBody(exchange) 요청 본문 파싱
 *   |   | authService.login(...) AuthService.login() 호출
 *   |   |   | userRepository.findByUsername(...) JdbcUserRepository가 실제 DB 조회
 *   |   |   | BCrypt.checkpw(...) 비밀번호 검증
 *   |   |   | JwtUtil.createToken(user) 로그인 성공 시 JWT 토큰 발급
 *   |   |   | writeJson / redirect 등 응답 전송
 * */

```

- auth0이라는 라이브러리는 JWT(JSON Web Token)을 발행하거나 웹 인증 관련 외부 유틸 라이브러리
- 순수 자바에서도 기본적으로 간이 HTTP 서버를 제공하지만, 서비스용으로는 부적합. (동시성, 보안등)
- 문자열을 암호화하는 라이브러리 중 해당 프로젝트에서는 BCrypt를 사용.
- 그 외에도 자바에서 기본적으로 지원하는 sql(JDBC 표준 API), time, util 라이브러리를 사용.

참고사항

```

dependencies {
    // MariaDB JDBC 드라이버
    implementation 'org.mariadb.jdbc:mariadb-java-client:3.5.6'
    // JWT 라이브러리 추가
    implementation 'com.auth0:java-jwt:4.4.0'
    // 비밀번호 해시용 jBCrypt

```

```
implementation 'org.mindrot:jbcrypt:0.4'
}
```

외부 라이브러리를 사용하기 위해서 build.gradle 파일의 종속성 부분에 사용할 라이브러리를 추가.

필드 살펴보기

```
// ===== DB CONFIG (환경에 맞게 수정) ===== //
private static final String DB_URL = "jdbc:mariadb://localhost:53301/sql_db";
private static final String DB_USER = "root";
private static final String DB_PASSWORD = "SqlDb-a1";

// 여기서는 "세션ID 문자열" -> "그 세션에 로그인된 User 객체"를 저장하는 용도로 씁니다.
// ConcurrentHashMap을 쓰는 이유: 여러 사용자의 요청이 동시에 여러 스레드에서 처리되는데
// (아래 main()의 newFixedThreadPool(10) 참고), 일반 HashMap은 동시 수정 시 데이터가 깨질 수 있어
// 스레드에 안전한 ConcurrentHashMap을 사용합니다.
private static final Map<String, User> SESSION_STORE = new ConcurrentHashMap<>();
private static final String SESSION_COOKIE_NAME = "SESSION_ID";

// ===== JWT COOKIE 이름 ===== //
private static final String JWT_COOKIE_NAME = "ACCESS_TOKEN";

// ===== (선택) 프론트엔드(Vue 등)에서 "로그인 여부"만 간단히 확인하기 위한 쿠키 이름 ===== //
private static final String APP_AUTH_COOKIE_NAME = "APP_AUTH";

// ===== CORS (dev) ===== //
// CORS: 브라우저가 "다른 출처(origin)"로 보내는 요청을 기본적으로 막는 보안 정책(Same-Origin Policy)을
// 서버가 "이 출처는 허용한다"고 명시적으로 응답 헤더에 적어줌으로써 풀어주는 것.
// 지금은 프론트(5174번 포트)와 백엔드(8080번 포트)가 포트만 다른 "다른 출처"라서 필요합니다.
private static final String DEV_FRONTEND_ORIGIN = "http://localhost:5174";
```

- Day1에서 진행한 실습에서 설정한 DB를 사용
- 세션을 저장하는 Map을 만들어서 로그인된 User 객체를 가지고 있음.
- 쿠키 정보를 저장할 때 쓰일 이름을 설정.
- CORS 오류를 막기 위해 ORIGIN 주소를 설정.

클래스 살펴보기

User

```
// 데이터(필드)만 담고 있는 순수한 자바 객체. private 필드 + public getter/setter 조합을
// "캡슐화(encapsulation)"라고 부르며, 외부에서 필드를 직접 건드리지 못하게 막고
// 반드시 정해진 메소드를 통해서만 값을 읽고 쓰게 강제하는 설계 원칙입니다.
public static class User {
    private Long id; // Long: 기본형 long의 "객체 버전"(래퍼 클래스). null을 가질 수 있어 "아직 없음"을
    표현 가능
    private String username;
    private String password; // 주의: 여기 저장되는 값은 평문이 아니라 BCrypt 해시값 (아래 AuthService 참고)

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; } // this.id: "필드 id"를 가리킴 (매개변수 id와 이름이 같아
    구분 필요)

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

- 실제 데이터베이스를 이용할 때 사용할 데이터 클래스
- 캡슐화를 통해, 필드는 숨기고, 메소드를 통해 필드의 값을 제어.
- 파라미터와 필드의 이름이 같다면 **this 키워드**를 통해 객체의 필드에 접근할 수 있음.

```
// [interface - 인터페이스]
// 인터페이스는 "무엇을 할 수 있는지(메소드 목록)"만 정의하고, "어떻게 하는지(구현)"는 정의하지 않는
// 일종의 "계약서"입니다. AuthService는 UserRepository라는 계약(인터페이스)에만 의존하고,
// 그 뒤에 실제로 MariaDB를 쓰든, 나중에 다른 DB나 테스트용 가짜 구현으로 바꾸든 상관하지 않습니다.
// 이 원칙을 DIP(의존관계 역전 원칙)라고 하며, 스프링의 @Autowired 의존성 주입이 바로 이 위에서 동작합니다.
public interface UserRepository {
    void save(User user) throws Exception;
    User findByUsername(String username) throws Exception;
}
```

- 인터페이스를 통해 DB에 구애받지 않고 사용가능.
- 즉, 데이터를 저장하고 찾는 과정은 그냥 save(), findByUserName() 메소드 호출로 같은 결과를 얻을 수 있음.

```
// [implements - 인터페이스 구현]
// "UserRepository의 계약을 지키겠다"고 선언하고, save/findByUsername의 실제 동작(구현)을 작성합니다.
public static class JdbcUserRepository implements UserRepository {
    private final String url;
    private final String user;
    private final String password;

    // 생성자(constructor): new JdbcUserRepository(...)로 객체를 만들 때 딱 한 번 실행되며,
    // 필드 초기화를 담당합니다. 메소드처럼 보이지만 이름이 클래스명과 같고 반환 타입이 없습니다.
    public JdbcUserRepository(String url, String user, String password) {
        this.url = url;
        this.user = user;
        this.password = password;
    }

    private Connection getConnection() throws SQLException {
        // [설계 노트] 이 메소드가 호출될 때마다 "새로운 DB 연결"을 매번 새로 맺습니다(커넥션 풀 없음).
        // 연결을 맺고 끊는 과정 자체가 비용이 커서, 트래픽이 조금만 늘어도 성능 병목이 됩니다.
        // 스프링부트는 HikariCP 같은 커넥션 풀을 기본 내장해서, 미리 만들어진 연결을 재사용합니다.
        return DriverManager.getConnection(url, user, password);
    }

    @Override // 이 메소드가 인터페이스(또는 부모 클래스)의 메소드를 "재정의"한 것임을 컴파일러에게 알림.
               // 오타 등으로 실제로는 재정의가 아닌데 @Override를 붙이면 컴파일 에러가 나서 실수를 잡아줌.
    public void save(User u) throws Exception {
        String sql = "INSERT INTO users(username, password) VALUES(?, ?)";
        // PreparedStatement: SQL문에 값을 "문자열 이어붙이기"가 아니라 ? 자리표시자로 바인딩함으로써
        // SQL 인젝션 공격을 원천 차단하는 방식. 반드시 이 방식을 써야 합니다.
        // try-with-resources에 자원을 콤마로 여러 개 나열하면, 선언의 "역순"으로 자동 close됩니다.
        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS))
        {
            ps.setString(1, u.getUsername()); // 1번째 ? 자리에 값 바인딩 (인덱스는 1부터 시작)
            ps.setString(2, u.getPassword());
            ps.executeUpdate(); // INSERT/UPDATE/DELETE 실행 (영향받은 행 수를 반환하지만 여기선 안 씀)
            // RETURN_GENERATED_KEYS로 요청했으므로, DB가 자동 채번한 AUTO_INCREMENT id 값을 꺼내옴
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) { // ResultSet은 커서 방식: next()가 "다음 행이 있으면 이동 후 true" 반환
                    u.setId(rs.getLong(1)); // 첫 번째 컬럼 값을 long으로 읽어옴
                }
            }
        }
        // try 블록이 끝나면 ps -> conn 순서로 자동 close (선언의 역순)
    }

    @Override
    public User findByUsername(String username) throws Exception {
        String sql = "SELECT id, username, password FROM users WHERE username = ?";
        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS))
        {
            ps.setString(1, username);
            ps.executeQuery();
            if (ps.next()) {
                User user = new User();
                user.setId(rs.getLong(1));
                user.setUsername(rs.getString(2));
                user.setPassword(rs.getString(3));
                return user;
            }
        }
        return null;
    }
}
```

```

        PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, username);
    try (ResultSet rs = ps.executeQuery()) { // SELECT는 executeQuery (결과 집합 반환)
        if (rs.next()) { // 결과가 있으면 (사용자를 찾았으면)
            User u = new User();
            u.setId(rs.getLong("id")); // 컬럼명으로 값 조회
            u.setUsername(rs.getString("username"));
            u.setPassword(rs.getString("password"));
            return u;
        }
    }
    return null; // 결과가 없으면 (해당 username의 사용자가 없으면) null 반환
}
}
}

```

- UserRepository의 실제 구현 클래스
- save(), findByUserName() 함수를 조건에 맞게 구현 수 있도록 작성.

```

// [Service 계층]
// "업무 규칙(비즈니스 로직)"을 담당하는 클래스. DB 접근 방법(JDBC)은 UserRepository에게 위임하고,
// 여기서는 "회원가입 시 중복 체크를 한다", "로그인 시 비밀번호를 검증한다" 같은 규칙만 다룹니다.
public static class AuthService {
    private final UserRepository userRepository;

    // [생성자 주입(Constructor Injection)]
    // main()에서 new AuthService(userRepository)로 호출될 때, 미리 만들어진
    // JdbcUserRepository 객체가 이 필드에 "주입"됩니다. AuthService 입장에서는 그게
    // JdbcUserRepository인지 몰라도 되고, UserRepository 계약만 지켜지면 됩니다 (다형성).
    public AuthService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // 회원가입
    public void signUp(String username, String rawPassword) throws Exception {
        User existing = userRepository.findByUsername(username); // ↑
        JdbcUserRepository.findByUsername 호출
        if (existing != null) {
            // [설계 노트] SQL 전용 예외 클래스를 "이미 존재하는 사용자"라는 업무 규칙 위반을 표현하는 데
            // 재활용하고 있습니다. 의미상 어색하고(DB 에러가 아니라 업무 규칙 위반인데), 호출하는 쪽에서
            // "SQL 예외인 척하는 업무 예외"를 구분해서 잡아야 하는 불편함이 있습니다.
            // 스프링부트에서는 DuplicateUsernameException 같은 커스텀 예외 + @ControllerAdvice로
            // 훨씬 명확하고 일관되게 처리합니다.
            throw new SQLIntegrityConstraintViolationException("username already exists");
        }

        // BCrypt.hashpw(원문, gensalt(강도)) : 비밀번호를 "단방향 해시"로 변환.
        // 해시는 원래 비밀번호로 되돌릴 수 없고(단방향), 같은 비밀번호라도 매번 다른 결과가 나오도록
        // salt(무작위 값)를 섞습니다. gensalt(12)의 12는 "얼마나 느리게(안전하게) 계산할지"의 강도(cost factor).
        String hashed = BCrypt.hashpw(rawPassword, BCrypt.gensalt(12));

        User u = new User();
        u.setUsername(username);
        u.setPassword(hashed); // 평문이 아니라 해시값을 저장 -> DB가 털려도 원문 비밀번호는 알 수 없음
        userRepository.save(u); // ↑ JdbcUserRepository.save 호출
    }

    public User login(String username, String rawPassword) throws Exception {
        User u = userRepository.findByUsername(username);
        if (u == null) {
            return null; // 그런 사용자 자체가 없음
        }

        // BCrypt.checkpw(입력한 원문, DB에 저장된 해시) : 내부적으로 DB 해시에 박혀있는 salt를 꺼내
        // 입력값을 같은 방식으로 다시 해시해서 두 값을 비교합니다. (해시를 복호화하는 게 아님!)
        if (!BCrypt.checkpw(rawPassword, u.getPassword())) {
            return null; // 비밀번호 불일치
        }
    }
}

```

```

    }

    return u; // 로그인 성공 -> 이 User 객체가 호출부(handler)로 돌아가 세션/JWT 발급에 쓰임
}
}

```

- 실제 비즈니스 로직으로, 로그인을 처리하는 클래스.
- 해당 클래스는 JdbcUserRepository 객체가 뭔지 모르고, 그냥 UserRepository의 함수 save(), findByUserName()만 호출하면 원하는 값을 반환
- 인터페이스가 왜 사용되는지 잘 보여줌.

```

// ===== JWT 유틸 =====
// static 메소드만 모아둔 "유틸리티 클래스"라서, 굳이 new JwtUtil()로 객체를 만들 필요 없이
// JwtUtil.createToken(...)처럼 클래스 이름으로 바로 호출합니다.
public static class JwtUtil {
    // [설계 노트] 실제 서비스에서는 이 비밀키를 코드에 두지 않고 환경변수/설정 파일로 분리해야 합니다.
    // 이 키가 유출되면 누구나 위조된 로그인 토큰을 만들 수 있게 되어 심각한 보안 사고로 이어집니다.
    private static final String SECRET = "RANDOM_SECRET_KEY";
    // HMAC256: "대칭키" 방식의 서명 알고리즘. 토큰을 만들 때 쓴 키와 검증할 때 쓰는 키가 동일합니다.
    private static final Algorithm ALG = Algorithm.HMAC256(SECRET);
    private static final String ISSUER = "simple-auth-server"; // 이 토큰을 "누가 발급했는지" 표시하는 값

    public static String createToken(User user) {
        Instant now = Instant.now(); // 현재 시각(UTC 기준 타임스탬프)을 얻음
        // JWT.create()...sign(ALG) : 메소드 체이닝(각 메소드가 자기 자신을 반환해서 .으로 계속 이어 부를 수
        // 있게 만든 패턴, "빌더 패턴"). 토큰에 담을 정보(클레임)들을 하나씩 쌓다가 마지막에 서명해서 완성.
        return JWT.create()
            .withIssuer(ISSUER)
            .withIssuedAt(Date.from(now)) // 발급 시각
            .withExpiresAt(Date.from(now.plus(1, ChronoUnit.HOURS))) // 발급 시각 + 1시간 뒤 만료
            .withSubject(String.valueOf(user.getId())) // 토큰의 "주인"을 문자열로 변환해서 저장
            .withClaim("username", user.getUsername()) // 커스텀 데이터 추가
            .sign(ALG); // 지금까지 쌓은 내용을 SECRET 키로 서명해서 최종 토큰 문자열 생성
    }

    // 서버가 쿠키로 받은 토큰이 "우리가 발급한 게 맞는지, 위조되지 않았는지, 만료되지 않았는지" 검증
    public static DecodedJWT verifyToken(String token) throws JWTVerificationException {
        JWTVerifier verifier = JWT.require(ALG)
            .withIssuer(ISSUER) // 우리가 지정한 issuer와 다르면 검증 실패로 처리
            .build();
        return verifier.verify(token); // 서명/issuer/만료시간을 모두 검사 후, 문제 없으면 내용을 돌려줌
    }
}

```

- JWT를 쉽게 생성하기 위한 유틸 클래스
- 생성은 빌더 패턴을 이용해서 만들었는데, 이는 다음 강의에서 배울 내용.
- 또한 서버가 쿠키로 받은 토큰이 알맞은 것인지 검증하는 기능도 제공.

main 살펴보기

```

public static void main(String[] args) throws Exception {

    // Class.forName(...) : 문자열로 된 클래스 이름을 런타임에 강제로 "로딩"시키는 리플렉션 기법.
    // 옛날 JDBC 드라이버들은 이렇게 드라이버 클래스를 한 번 로딩해줘야 DriverManager가 인식했습니다.
    // (최신 드라이버는 자동 등록(SPI) 기능이 있어 사실 생략 가능한 경우가 많습니다.)
    Class.forName("org.mariadb.jdbc.Driver");

    // ---- 객체 조립(수동 의존성 주입, manual DI) ----
    // 1) DB와 직접 통신하는 객체를 만들고
    UserRepository userRepository = new JdbcUserRepository(DB_URL, DB_USER, DB_PASSWORD);
    // 2) 그 객체를 AuthService의 "생성자 매개변수로 넘겨서(주입해서)" AuthService를 만듭니다.
    // AuthService는 "DB를 어떻게 조회하는지"는 몰라도 되고, "로그인 판단 로직"에만 집중합니다.
    // 이렇게 역할을 나누는 것을 관심사 분리(Separation of Concerns)라고 합니다.
}

```

```

AuthService authService = new AuthService(userRepository);

// ---- HTTP 서버 생성 ----
// new InetSocketAddress(8080) : "이 컴퓨터의 8080번 포트에서 요청을 기다리겠다"는 주소 정보.
// 두 번째 인자 0은 backlog(대기 큐 크기)로, 0이면 시스템 기본값을 사용합니다.
HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);

// Executors.newFixedThreadPool(10) : 동시에 최대 10개의 요청을 병렬로 처리할 수 있는
// "작업자 스레드 10개짜리 팀"을 만듭니다. 요청이 10개를 넘으면 앞선 요청이 끝날 때까지 대기합니다.
// setExecutor(...)로 이 스레드풀을 서버에 연결하면, 각 요청이 이 풀의 스레드 중 하나에서 처리됩니다.
server.setExecutor(Executors.newFixedThreadPool(10));

// =====
// [server.createContext(경로, 핸들러) - 라우팅 등록]
//
// createContext의 두 번째 인자는 HttpHandler라는 "인터페이스" 타입입니다.
// HttpHandler는 딱 하나의 추상 메소드 void handle(HttpExchange exchange)만 가지고 있는데,
// 이렇게 메소드가 딱 하나뿐인 인터페이스를 "함수형 인터페이스"라고 부릅니다.
//
// exchange -> { ... } 는 "람다식(lambda expression)"입니다.
// 원래는 아래처럼 익명 클래스로 길게 써야 할 코드를
//
// new HttpHandler() {
//     public void handle(HttpExchange exchange) throws IOException {
//         ...
//     }
// }
//
// 자바 8부터는 화살표(->)를 이용해 "이 파라미터(exchange)가 들어오면 이 코드를 실행해라"라고
// 훨씬 짧게 쓸 수 있게 해준 것이 람다식입니다. 즉 아래 각 createContext 블록은 전부
// "이 URL 경로로 요청이 오면, 이 handle 메소드를 실행해라"를 등록하는 코드입니다.
// =====

// GET / -> /login 리다이렉트
server.createContext("/", exchange -> {
    // exchange.getRequestMethod() : 이 요청이 GET인지 POST인지 등을 문자열로 반환.
    // equalsIgnoreCase : 대소문자를 무시하고 문자열이 같은지 비교 (== 대신 써야 하는 이유:
    // 자바에서 문자열은 ==으로 비교하면 "내용"이 아니라 "객체 참조"를 비교하게 됩니다).
    if ("GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        redirect(exchange, "/login");
    } else {
        methodNotAllowed(exchange);
    }
});

// GET /login -> login.html 정적 파일 응답
server.createContext("/login", exchange -> {
    if ("GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        serveResource(exchange, "web/login.html", "text/html; charset=utf-8");
    } else {
        methodNotAllowed(exchange);
    }
});

// GET /signup -> signup.html
server.createContext("/signup", exchange -> {
    if ("GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        serveResource(exchange, "web/signup.html", "text/html; charset=utf-8");
    } else {
        methodNotAllowed(exchange);
    }
});

// GET /static/* -> CSS/JS 등 정적 리소스 응답
server.createContext("/static", exchange -> {
    if (!"GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        methodNotAllowed(exchange);
        return; // return만 쓰면 이 handle 메소드를 즉시 종료(뒤 코드 실행 안 함)
    }
}

```

```

String path = exchange.getRequestURI().getPath(); // 예: /static/style.css
// replaceFirst(정규식, 대체문자열): "/static/" 또는 "/static"으로 시작하는 부분을 지워서
// 실제 파일명만 남깁니다. (?는 앞 문자가 있어도 되고 없어도 된다는 정규식 기호)
String filename = path.replaceFirst("/static/?", "");
if (filename.isEmpty()) {
    notFound(exchange);
    return;
}

String resourcePath = "web/" + filename; // 문자열 + 연산자: 문자열 이어붙이기(concatenation)
String contentType;
// if-else if-else 체인: 파일 확장자에 따라 브라우저에게 알려줄 MIME 타입을 다르게 지정
if (filename.endsWith(".css")) {
    contentType = "text/css; charset=utf-8";
} else if (filename.endsWith(".js")) {
    contentType = "application/javascript; charset=utf-8";
} else {
    contentType = "application/octet-stream"; // 알 수 없는 타입은 "그냥 바이너리"로 취급
}
serveResource(exchange, resourcePath, contentType);
});

// POST /api/signup -> 회원가입
server.createContext("/api/signup", exchange -> {
    if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
        methodNotAllowed(exchange);
        return;
    }
    // 아래 authService, userRepository처럼 main() 안의 지역 변수를 람다 밖에서 참조하는 것을
    // "클로저(closure)"라고 합니다. 자바는 람다가 캡처하는 지역 변수가 사실상 final(effectively
    // final, 즉 이후에 재대입되지 않는 변수)이어야 한다는 제약이 있습니다.
    Map<String, String> params = parseFormBody(exchange);
    String username = params.get("username"); // Map.get(key): 키에 해당하는 값을 꺼냄 (없으면 null)
    String password = params.get("password");

    if (isBlank(username) || isBlank(password)) {
        writeText(exchange, 400, "username and password are required");
        return;
    }

    // try-catch: "이 블록(try) 안에서 예외가 나면, 그 예외 종류에 맞는 catch 블록으로 점프해라"
    try {
        authService.signUp(username, password);
        redirect(exchange, "/login");
    } catch (SQLIntegrityConstraintViolationException dup) {
        // DB의 UNIQUE 제약(username 중복)을 어겼을 때 JDBC가 던지는 구체적인 예외 타입
        writeText(exchange, 400, "username already exists");
    } catch (Exception e) {
        // 위에서 못 잡은 나머지 모든 예외를 여기서 포괄적으로 처리 (catch는 위에서부터 순서대로 검사됨)
        e.printStackTrace(); // 콘솔에 예외 상세 내역(스택 트레이스) 출력 - 디버깅용
        writeText(exchange, 500, "signup error");
    }
});

// POST /api/login
// -> 로그인 + 세션 발급 + JWT 발급 + 프론트엔드로 리다이렉트
server.createContext("/api/login", exchange -> {
    if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
        methodNotAllowed(exchange);
        return;
    }

    Map<String, String> params = parseFormBody(exchange);
    String username = params.get("username");
    String password = params.get("password");

    if (isBlank(username) || isBlank(password)) {
        writeText(exchange, 400, "username and password are required");
        return;
    }

```

```

    }

    try {
        // ↓↓↓ 실제 호출 흐름: AuthService.login() 내부에서 다시
        //     userRepository.findByUsername()과 BCrypt.checkpw()를 호출합니다.
        //     (아래 AuthService 클래스 정의부에서 자세히 설명)
        User user = authService.login(username, password);
        if (user == null) {
            writeText(exchange, 401, "invalid credentials");
            return;
        }

        // 1) 세션 쿠키 발급
        // UUID.randomUUID() : 전 세계에서 겹칠 확률이 사실상 0에 가까운 무작위 문자열 생성
        String sessionId = UUID.randomUUID().toString();
        SESSION_STORE.put(sessionId, user); // Map.put(key, value): 저장
        // 응답 헤더에 Set-Cookie를 추가하면, 브라우저가 이후 요청마다 이 쿠키를 자동으로 실어 보냄.
        // HttpOnly: 자바스크립트(document.cookie)로 이 쿠키를 읽지 못하게 막아 XSS 공격을 방어.
        exchange.getResponseHeaders().add(
            "Set-Cookie",
            SESSION_COOKIE_NAME + "=" + sessionId + "; Path=/; HttpOnly; SameSite=Lax"
        );

        // 2) JWT 쿠키 발급 (아래 JwtUtil.createToken 설명 참고)
        String jwt = JwtUtil.createToken(user);
        exchange.getResponseHeaders().add(
            "Set-Cookie",
            JWT_COOKIE_NAME + "=" + jwt + "; Path=/; HttpOnly; SameSite=Lax"
        );

        // 3) 프론트에서 "로그인 여부"만 자바스크립트로 읽을 수 있도록 HttpOnly 없이 발급
        exchange.getResponseHeaders().add(
            "Set-Cookie",
            APP_AUTH_COOKIE_NAME + "=1; Path=/; SameSite=Lax"
        );

        // 4) 로그인 후 이동할 페이지로 리다이렉트
        redirect(exchange, "/home.html");
        // redirect(exchange, "http://localhost:5174/intro_main.html");

    } catch (Exception e) {
        e.printStackTrace();
        writeText(exchange, 500, "login error");
    }
});

// GET /api/auth/check -> 세션 또는 JWT로 로그인 여부 확인
server.createContext("/api/auth/check", exchange -> {
    // OPTIONS: 브라우저가 실제 요청 전에 "이 요청 보내도 되는지" 미리 물어보는 CORS Preflight 요청
    if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
        addCorsHeaders(exchange);
        exchange.sendResponseHeaders(204, -1); // 204 No Content: 본문 없이 "허용됨"만 응답
        exchange.close();
        return;
    }

    if (!"GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        methodNotAllowed(exchange);
        return;
    }

    addCorsHeaders(exchange);

    // (A) 세션 방식 검사
    String sessionId = getCookieValue(exchange, SESSION_COOKIE_NAME);
    User sessionUser = null;
    if (!isBlank(sessionId)) {
        sessionUser = SESSION_STORE.get(sessionId); // 없으면 null 반환
    }

```

```

// (B) JWT 방식 검사
String token = getCookieValue(exchange, JWT_COOKIE_NAME);
boolean jwtOk = false;
String jwtUsername = null;
String jwtSubject = null;

if (!isBlank(token)) {
    try {
        // JwtUtil.verifyToken 내부에서 서명 위조 여부/발급자(issuer)/만료시간(exp)을 검사함
        DecodedJWT decoded = JwtUtil.verifyToken(token);
        jwtOk = true;
        // getClaim("username") : 토큰 안에 넣어둔 커스텀 데이터(클레임)를 꺼냄
        jwtUsername = decoded.getClaim("username").asString();
        jwtSubject = decoded.getSubject(); // 토큰의 주인(sub) - 여기선 user.getId()를 넣었었음
    } catch (JWTVerificationException ex) {
        // 서명이 다르거나(위조), 만료되었거나(exp 지남) 등 "검증 실패"의 표준 예외
        jwtOk = false;
    } catch (Exception ex) {
        jwtOk = false;
    }
}

// 정책: 세션 OR JWT 둘 중 하나라도 유효하면 로그인 상태로 인정
if (sessionUser == null && !jwtOk) {
    writeJson(exchange, 401, "{\"ok\":false,\"reason\":\"UNAUTHORIZED\"}");
    return;
}

// 삼항 연산자 대신 if-else로 두 경우(세션 우선, 그 다음 JWT)에 따라 다른 JSON 응답 생성
if (sessionUser != null) {
    String body = "{\"ok\":true,\"via\":\"session\",\"username\":\""
        + escapeJson(sessionUser.getUsername()) + "\"}";
    writeJson(exchange, 200, body);
} else {
    String body = "{\"ok\":true,\"via\":\"jwt\",\"username\":\""
        + escapeJson(jwtUsername) + "\",\"sub\":\""
        + escapeJson(jwtSubject) + "\"}";
    writeJson(exchange, 200, body);
}
});

server.createContext("/home.html", exchange -> {
    if (!"GET".equalsIgnoreCase(exchange.getRequestMethod())) {
        methodNotAllowed(exchange);
        return;
    }
    serveResource(exchange, "web/home.html", "text/html; charset=utf-8");
});

// POST /api/logout -> 로그아웃 (세션/쿠키 전부 무효화)
server.createContext("/api/logout", exchange -> {
    if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
        methodNotAllowed(exchange);
        return;
    }
    String sessionId = resolveSessionId(exchange);
    if (sessionId != null) {
        SESSION_STORE.remove(sessionId); // 서버 메모리에서 세션 삭제
        // Max-Age=0으로 같은 이름의 쿠키를 내려보내면 브라우저가 즉시 해당 쿠키를 삭제함
        exchange.getResponseHeaders().add(
            "Set-Cookie",
            SESSION_COOKIE_NAME + "=: Path=/; Max-Age=0; Path=/"
        );
    }

    exchange.getResponseHeaders().add(
        "Set-Cookie",
        JWT_COOKIE_NAME + "=: Path=/; Max-Age=0;"
    );
});

```

```

    );

    exchange.getResponseHeaders().add(
        "Set-Cookie",
        APP_AUTH_COOKIE_NAME + "; Path=/; Max-Age=0;"
    );

    redirect(exchange, "/login");
});

server.start(); // 지금까지 등록한 라우팅으로 실제 요청을 받기 시작 (이 줄 전까지는 아무 요청도 안 받음)
System.out.println("Server started at http://localhost:8080");
}

```

- JdbcUserRepository, AuthService 인스턴스를 생성
- **HttpServer server**를 8080포트로 간이 서버를 실행해서 테스트를 진행.
- GET/POST API를 통해 세부내용 처리(로그인, 회원가입, 메인화면)

리소스 응답 헬퍼 메소드 살펴보기

```

// ===== 리소스/응답 헬퍼 메소드들 =====
// 아래 메소드들은 "HTTP 응답을 어떻게 만들어 내려보낼지"의 반복되는 패턴을 한 곳에 모아둔 것입니다.
// 이렇게 공통 로직을 메소드로 뽑아두면(중복 제거), 위의 각 handler 람다들은 짧게 유지됩니다.

private static void serveResource(HttpExchange exchange, String resourcePath, String contentType)
throws IOException {
    // getClassLoader().getResourceAsStream(...) : jar 파일 내부에 포함된 리소스 파일(HTML 등)을
    // 파일 시스템 경로가 아니라 "클래스패스" 기준으로 읽어옴 (배포 시 jar 안에 들어있어도 동작하게 하기 위함)
    InputStream is = SimpleAuthServer.class.getClassLoader().getResourceAsStream(resourcePath);
    if (is == null) {
        notFound(exchange);
        return;
    }
    byte[] bytes = is.readAllBytes(); // 스트림 전체를 바이트 배열로 한 번에 읽음
    exchange.getResponseHeaders().add("Content-Type", contentType);
    // sendResponseHeaders(상태코드, 본문 길이) : 이 호출 이후에야 실제로 응답 헤더가 전송됨
    exchange.sendResponseHeaders(200, bytes.length);
    // try-with-resources: 괄호 안에서 연 자원(OutputStream)을 try 블록이 끝나면
    // (정상 종료든 예외든) 자동으로 close()까지 해주는 문법. close() 깜빡하는 실수를 방지함.
    try (OutputStream os = exchange.getResponseBody()) {
        os.write(bytes);
    }
}

private static void writeText(HttpExchange exchange, int status, String body) throws IOException {
    byte[] bytes = body.getBytes(StandardCharsets.UTF_8); // 문자열을 UTF-8 바이트로 변환
    exchange.getResponseHeaders().add("Content-Type", "text/plain; charset=utf-8");
    exchange.sendResponseHeaders(status, bytes.length);
    try (OutputStream os = exchange.getResponseBody()) {
        os.write(bytes);
    }
}

private static void writeJson(HttpExchange exchange, int status, String json) throws IOException {
    // [설계 노트] 실제로는 문자열을 "직접 이어붙여" JSON을 만들고 있습니다(escapeJson 참고).
    // 필드가 늘어나거나 값 안에 특수문자가 섞이면 JSON이 깨지기 쉬운 취약한 방식입니다.
    // 스프링부트에서는 Jackson(ObjectMapper)이 객체를 자동으로 안전하게 JSON 문자열로 변환해줍니다.
    byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
    exchange.getResponseHeaders().add("Content-Type", "application/json; charset=utf-8");
    exchange.sendResponseHeaders(status, bytes.length);
    try (OutputStream os = exchange.getResponseBody()) {
        os.write(bytes);
    }
}

```

```

private static void redirect(HttpExchange exchange, String location) throws IOException {
    exchange.getResponseHeaders().add("Location", location); // 브라우저에게 "여기로 다시 요청해라"
    exchange.sendResponseHeaders(302, -1); // 302: 임시 리다이렉트, 본문 길이 -1은 "본문 없음"
    exchange.close();
}

private static void methodNotAllowed(HttpExchange exchange) throws IOException {
    writeText(exchange, 405, "Method Not Allowed");
}

private static void notFound(HttpExchange exchange) throws IOException {
    writeText(exchange, 404, "Not Found");
}

// application/x-www-form-urlencoded 형식의 요청 본문(예: username=abc&password=123)을
// Map<String, String> 형태로 직접 파싱하는 메소드.
// [설계 노트] 스프링부트에서는 @RequestParam이나 DTO 클래스 하나로 이 파싱이 전부 자동 처리됩니다.
private static Map<String, String> parseFormBody(HttpExchange exchange) throws IOException {
    String body = new String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
    Map<String, String> params = new HashMap<>(); // <>는 "다이아몬드 연산자": 좌변 타입으로 제네릭 타입 추론
    // String.split("&") : "&" 문자를 기준으로 문자열을 잘라 배열로 만들
    for (String pair : body.split("&")) { // 향상된 for문(for-each): 배열/컬렉션을 하나씩 꺼내며 순회
        if (pair.isEmpty()) continue; // continue: 이번 반복만 건너뛰고 다음 반복으로
        String[] kv = pair.split("=", 2); // 2: "="를 기준으로 최대 2조각까지만 자름 (값에 =가 있어도 안전)
        String key = urlDecode(kv[0]);
        String value = kv.length > 1 ? urlDecode(kv[1]) : ""; // 삼항 연산자: 조건 ? 참일때값 : 거짓일때값
        params.put(key, value);
    }
    return params;
}

private static String urlDecode(String s) {
    return URLDecoder.decode(s, StandardCharsets.UTF_8);
}

private static boolean isBlank(String s) {
    // null 체크를 먼저 하고(||는 왼쪽이 true면 오른쪽은 평가조차 안 하는 short-circuit 연산자이므로
    // s가 null이어도 s.trim()에서 NullPointerException이 나지 않음), trim() 후 비어있는지 검사
    return s == null || s.trim().isEmpty();
}

// 쿠키 문자열(예: "SESSION_ID=abc; ACCESS_TOKEN=xyz")에서 원하는 이름의 값만 뽑아내는 공통 함수.
// 세션 검사와 JWT 검사 둘 다 이 함수를 재사용합니다 (코드 중복 제거).
private static String getCookieValue(HttpExchange exchange, String name) {
    List<String> cookies = exchange.getRequestHeaders().get("Cookie");
    if (cookies == null) return null;
    for (String header : cookies) {
        // split(";\\s*") : 세미콜론 뒤에 공백이 0개 이상 있어도 구분자로 인식 (정규식, \\s는 공백문자)
        String[] parts = header.split(";\\s*");
        for (String part : parts) {
            String[] kv = part.split("=", 2);
            if (kv.length == 2 && name.equals(kv[0])) {
                // "문자열".equals(변수) 순서로 쓰면, 변수가 null이어도 NPE 없이 안전하게 false 반환됨
                return kv[1];
            }
        }
    }
    return null;
}

private static String resolveSessionId(HttpExchange exchange) {
    return getCookieValue(exchange, SESSION_COOKIE_NAME);
}

private static User resolveUser(HttpExchange exchange) {
    String sessionId = resolveSessionId(exchange);
    if (sessionId == null) return null;
    return SESSION_STORE.get(sessionId);
}

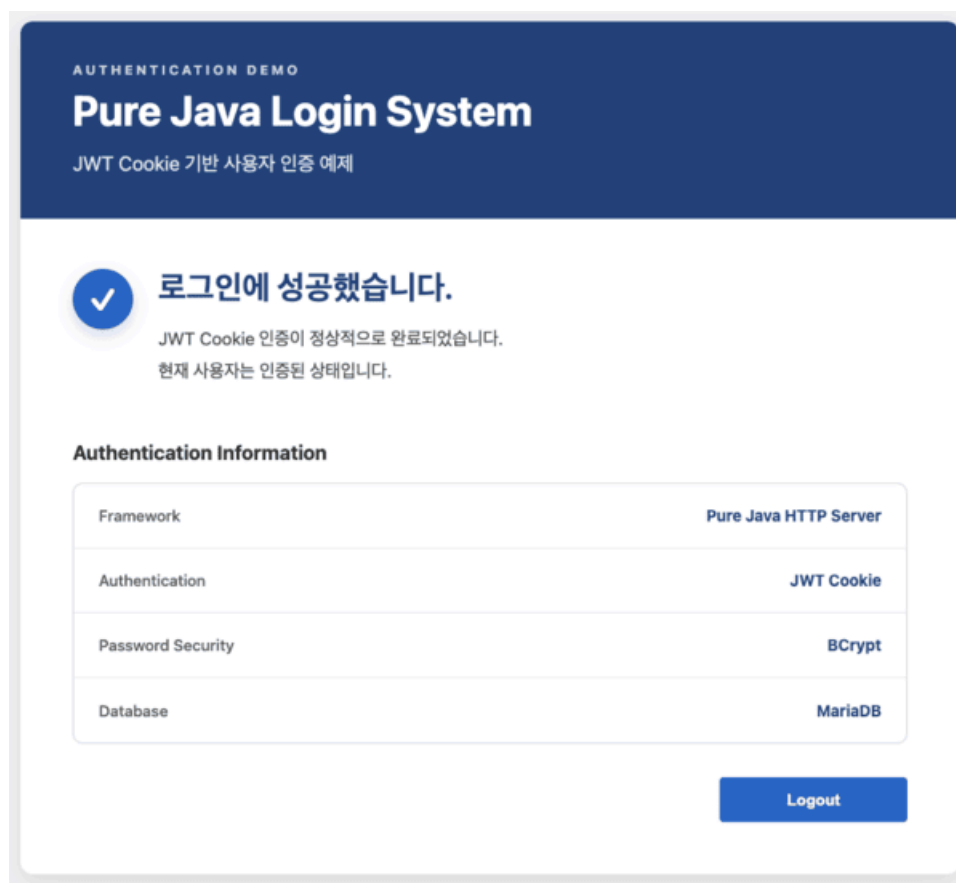
```

```
private static String escapeHtml(String s) {
    if (s == null) return "";
    // 문자열.replace(찾을문자, 바꿀문자) 체이닝: & < > " ' 를 HTML 엔티티로 바꿔 XSS(스크립트 삽입) 방지
    return s.replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("\"", "&quot;")
        .replace("'", "&#39;");
}

// JSON 문자열 안에 큰따옴표나 줄바꿈이 그대로 들어가면 JSON 형식이 깨지므로 최소한으로 이스케이프 처리
private static String escapeJson(String s) {
    if (s == null) return "";
    return s.replace("\\", "\\")
        .replace("\"", "\\")
        .replace("\n", "\\n")
        .replace("\r", "\\r")
        .replace("\t", "\\t");
}
```

- HTTP 응답을 어떻게 만들고 보낼지에 대해 반복되는 패턴을 미리 만들어 둔 메소드
- 이러한 공통 로직을 메소드로 뽑아두면 각 handler 람다들은 짧게 유지가능

3. 코드 결과



- 제공된 코드를 통해 실제로 서버를 띄우고, 회원가입과 로그인을 진행.
- 정상적으로 메인화면에 진입.

SCHEMAS

Filter objects

- sql_db
 - Tables
 - users
 - Columns
 - Indexes
 - Foreign Keys
 - Triggers
 - Views
 - Stored Procedures
 - Functions

1 • `SELECT * FROM sql_db.users;`

Limit to 1000 rows

100% 1:1

Result Grid Filter Rows: Search Edit: Export/Imp

id	username	password	created_at
1	java	java@test	2026-07-30 10...
4	admin	\$2a\$12\$YWqPCTADXBrla7DMcJU...	2026-07-31 13...
NULL	NULL	NULL	NULL

Object Info Session

Table: users

- DB의 Password 컬럼을 보면 기존의 평문 저장에서 암호화된 저장으로 변경된 것을 확인가능.